

情報科学演習 D

課題 4

担当教員 : 梶井 晃基
提出者 : 鶴見 俊介
所属/学年 : 基礎工学部 情報科学科 3 年 計算機科学コース
学籍番号 : 09B23057
電子メール : u299151f@ecs.osaka-u.ac.jp

提出日 : 2026 年 1 月 25 日
締切日 : 2026 年 1 月 30 日 23:59

1 システムの仕様

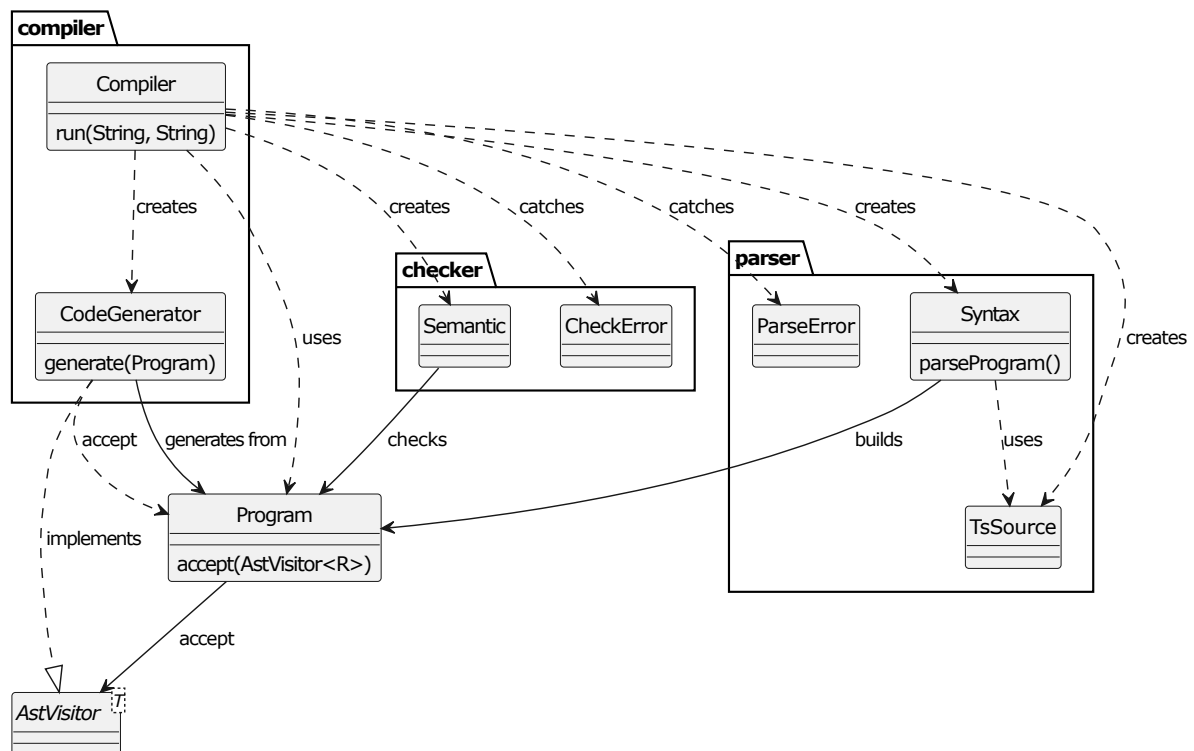
`Compiler.run(String, String)` は第一引数で指定した `ts` ファイルを読み込み、構文解析により `AST` を生成した後、意味解析で整合性を検査する。問題がなければ `CASL II` プログラムを生成して第二引数で指定した `cas` ファイルに出力する。出力時には生成コードの末尾に `data/cas/lib.cas` を連結し、実行可能な `CASL II` プログラムとする。入力に構文的または意味的誤りがある場合は、課題 2,3 と同形式のエラーメッセージを返して処理を終了する。入力ファイルが存在しない場合は `"File not found"` を返す。正常終了時は `"OK"` を返す。

2 課題達成の方針と設計

2.1 全体の処理の流れ

コンパイラのクラス図を図 1 に示す。本課題では、既存の構文・意味解析器を利用し、`AST` を中間表現として `CASL II` プログラム生成までを一貫して実行する。また、コード生成では `CodeGenerator` が `AstVisitor` を実装し、`AST` を走査して命令列を生成する (`Visitor` パターン)。

図 1: コンパイラのクラス図

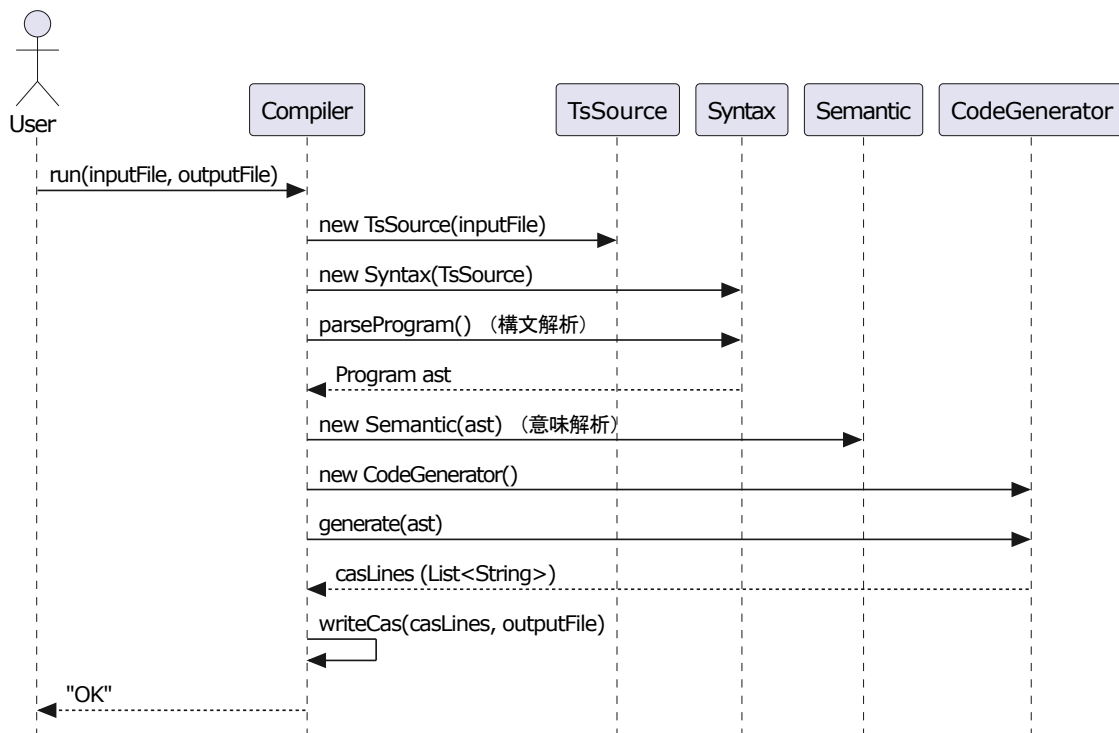


実行時の処理手順をまとめたものが図 2 のシーケンス図である。これに対応する処理の流れは次のとおりである。

1. `Compiler.run()` が呼び出され、入力 `ts` ファイルを対象としてコンパイルを開始する。
2. `TsSource` を生成して入力を読み込み、`Syntax` を生成する。
3. `Syntax.parseProgram()` を実行し、構文解析により `AST` を生成する。

4. Semantic を生成し、AST を入力として意味解析を行う。
5. CodeGenerator を生成し、generate(ast) により Visitor として Program.accept(this) を起点に AST を走査し、CASL II 命令列を生成する。
6. 生成した命令列を writeCas() で cas ファイルへ書き出し、末尾に lib.cas を連結して実行可能な CASL II プログラムとする。

図 2: コンパイラのシーケンス図



2.2 出力するコードの構成と出力方法

本課題では、`CodeGenerator.generate(Program)` が CASL II プログラム全体を組み立てる。図 3 に示すように、生成コードは (1) ヘッダ、(2) メイン部、(3) 手続き定義部、(4) データ領域、(5) フッタの順に配置する。`generate` 内では、初期化後に `emitHeader()` を出力し、`ast.accept(this)` を起点として `Visitor` でメイン部の命令列を生成する。メイン部の末尾には `RET` を付加し、その後に `procOut` に蓄積した各手続きの命令列を連結する。最後に `emitData()` で変数領域・文字列定数領域・`LIBBUF` を蓄積し、`emitFooter()` により `END` を蓄積して完結させる。

また、コード生成とファイル出力は分離して実装している。`CodeGenerator` は生成結果を逐次ファイルへ書き込むのではなく、まず命令列を文字列のリストとしてメモリ上に蓄積する。具体的には、`generate` 内で `out` (メイン出力) および `procOut` (手続き出力) に対して `emit()` を通じて行単位で命令列を追加し、最終的にヘッダ、メイン、手続き、データ、フッタをこの順に連結した `List<String>` を返す。

`Compiler.run()` は `generate` の戻り値として得られた `casLines` を `writeCas()` に渡し、ここで初めて `cas` ファイルとして書き出す。`writeCas()` では、生成した命令列に加えて `data/cas/lib.cas` を読み込み、その内容を末尾に追加して出力する。これにより、`MULT` や `DIV`、`WRTINT` などのライブラリルーチンを利用する前提の CASL II プログラムを、単体で実行可能な形で生成できる。

CASL	START	BEGIN	;	(1) Header
BEGIN	LAD	GR6,0	;	
	LAD	GR7,LIBBUF	;	
	...			(2) Main
	RET		;	
PROC0	NOP		;	(3) Procedures
	...			
	RET		;	
...				
VAR	DS	<varCount>	;	(4) Data
CHAR0	DC	'...'	;	
...				
LIBBUF	DS	256	;	
	END		;	(5) Footer

図 3: CASL II プログラムの構成

2.3 変数・配列・文字列定数の領域設計

- データ領域の配置方針

実行時に参照するデータはすべて CASL II のデータ領域としてプログラム末尾に集約し、変数領域 VAR, 文字列定数 CHAR<i>, 入出力バッファ LIBBUF をこの順に配置する。AST 走査中に収集した情報から、確保量 (varCount) やラベル (strLabel) を確定し、最後にまとめて出力する。

- VAR への一括割当てとオフセット管理

変数・配列・仮引数は VAR に連続確保し、識別子ごとに「VAR 先頭からのオフセット」を管理する。配列は要素数分を連続で確保し、後段の参照・代入を VAR + オフセットの形に統一する。

- 配列下限への対応

配列は下限が 0 とは限らないため、下限値を保持し、実行時に添字を補正して VAR 内の実オフセットへ変換する。これにより、下限の違いを意識せずに同一のアクセス規約で扱える。

- 文字列定数の共有と入出力規約への整合

文字列定数は重複定義を避けるために一元管理し、CHAR<i> としてデータ領域にまとめて配置する。さらに、入出力ライブラリが要求する形式に合わせて、文字列 (長さで先頭アドレス) と文字 (1 語) を区別して扱う。

- LIBBUF の確保

入出力ライブラリが利用する作業領域として LIBBUF を固定長で確保し、実行時にバッファ先頭アドレスをレジスタへ設定する前提でプログラム全体を構成する。

2.4 式評価と条件分岐

- 式評価の基本規約 (スタック受け渡し)

各 visit は「担当ノードを評価したら結果をスタックに PUSH して親へ返す」ことを共通規約とする。二項演算は left → right の順に accept して値を積み、POP した 2 値を用いて計算し、結果を再度 PUSH する。これにより、ネストした式でも評価順序が明確になり、生成処理をノード単位に局所化できる。

- 算術・論理演算の生成

ADD/SUB は ADDA/SUBA により命令列のみで完結し、計算結果を PUSH して式の値とする。単項演算も同様に、子式を評価して POP した値に変換を施して PUSH する。例えば NEG は 0 - x を SUBA で実現し、NOT は XOR ..., =#FFFF で #0000/#FFFF を反転させる。

- ライブラリ呼出による演算 (MUL/DIV/MOD)

MUL/DIV/MOD はライブラリルーチンに委譲する。左右オペランドを POP で取り出してレジスタに載せた後、CALL MULT や CALL DIV を発行し、戻り値レジスタの内容を PUSH して式の値として扱う。MOD は DIV 後の剰余レジスタ (実装では GR1) を PUSH して得る。

- 比較演算と真偽値生成 (#FFFF/#0000)

比較演算 (EQ, NEQ, LT, LE, GT, GE) は、POP した 2 値に対して比較命令を実行し、分岐命令により #FFFF (真) または #0000 (偽) を生成してスタックへ積む。比較が複数箇所に現れても衝突しないよう、jumpId を用いて TRUE<i>, BOTH<i> 等のラベルを一意に採番する。また、真偽値同士の比較は CPL, 整数同士の比較は CPA を用いるよう、varKind と exprKind を参照して isBoolCompare で切り替える。

- if/while の展開

条件式の値は #FFFF (真)・#0000 (偽) に正規化し、if と while は「#0000 なら分岐」という判定に統一する。visitIfS では ELSE<i>, ENDIF<i> を jumpId で採番し、条件判定後に偽なら else 側 (または末尾) へ分岐して合流点へ集約する。visitWhileS では LOOP<i>, ENDLP<i> を採番し、先頭で条件判定→偽なら終了→真なら本体生成→先頭へ戻る、の骨組みで展開する。図 4 は、if-else 文と while 文の分岐を実現している CASLII プログラムである。

... 条件式の計算 ...	LOOP<i> NOP
POP GR1	... 条件式の計算 ...
CPA GR1, =#0000	POP GR1
JZE ELSE<i>	CPA GR1, =#0000
... then 部 ...	JZE ENDLP<i>
JUMP ENDIF<i>	... body 部 ...
ELSE<i> NOP	JUMP LOOP<i>
... else 部 ...	ENDLP<i> NOP
ENDIF<i> NOP	

図 4: if-else 文 (左) および while 文 (右) の構成

2.5 手続きと引数の扱いとスコープ管理

- 手続き呼出の基本 (CALL/RET と引数の積み方)

手続きは CALL により本体へ分岐し、末尾の RET で呼出元へ復帰する。引数は呼出側で左から順に式を評価し、各値をスタックへ PUSH した後に CALL PROC<i> を発行する。

- 入口ラベル管理と出力構成

手続き名と入口ラベルの対応は procLabel で管理し、宣言時に PROC<i> を採番して手続き先頭ラベルとして確定する。生成した手続き本体の命令列はメイン部とは別に procOut に蓄積し、最後にメイン直後へ連結することで、「メイン → 手続き定義 → データ領域」という実行可能な配置を保つ。

- 仮引数処理 (スタック → VAR へのコピーと戻り番地の整形)

CALL 直後のスタックには引数と戻り番地が格納されているため、手続き開始直後に仮引数をスタックから取り出し、VAR 領域へコピーして以後は通常変数と同じ VAR+offset 規約で参照できるようにする。コピー後は、引数分だけ SP を進めて引数領域を破棄し、戻り番地のみがスタック先頭に残る形へ調整することで、手続き本体がスタックを一時領域として利用しても RET により確実に復帰できる状態を保証する。図 5 は、例として引数 2 つの手続き呼び出し操作におけるコード部分とスタックの様子を示したものである。

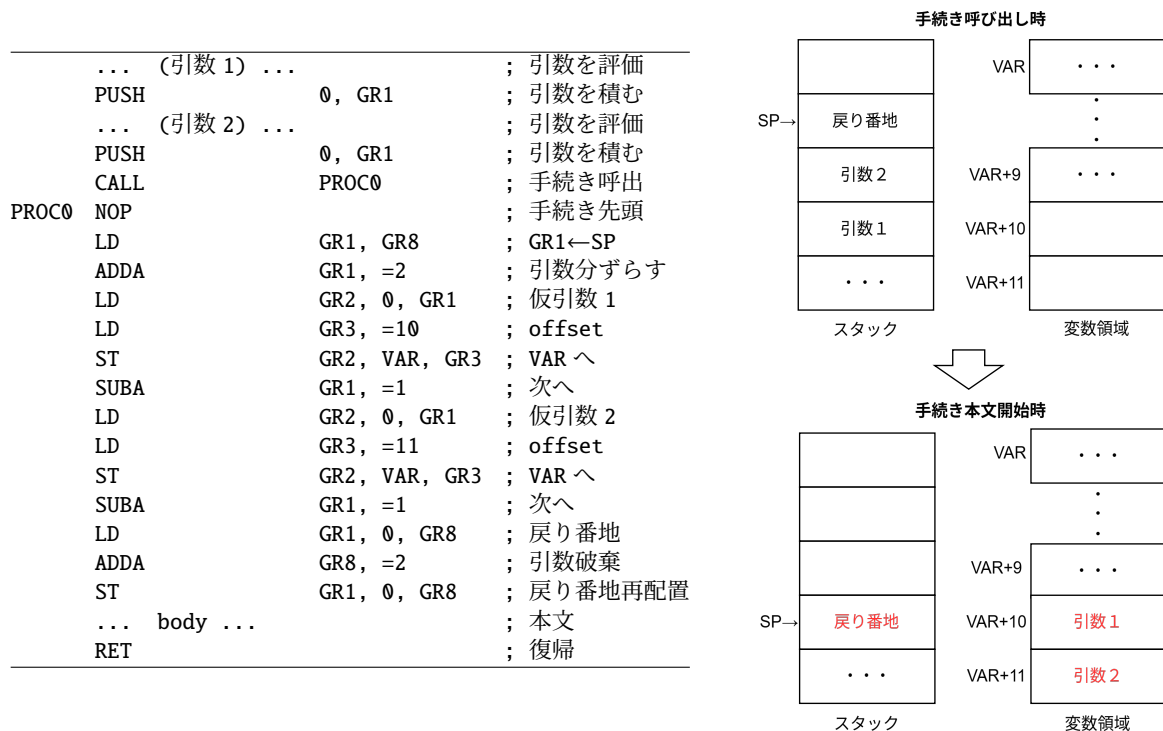
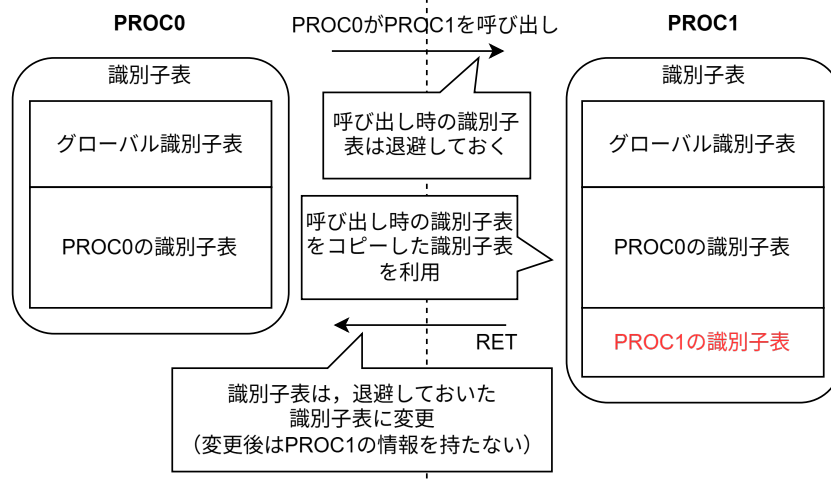


図 5: 引数 2 つの手続きの仮引数処理に伴うスタック操作：プログラム（左）とスタック状態（右）

・ スコープ管理（識別子表の退避・復元）

手続き内ではローカル変数・仮引数の登録により識別子表が一時的に拡張されるため、`varOffset`・`varKind`・`arrayLower` を手続き生成の開始時に退避し，終了時に復元する．これにより，手続き内で追加した識別子が外側へ漏れず，外側の同名識別子の隠蔽も含めて字句的スコープを維持できる．図 6 は，例として `PROC0` が `PROC1` を呼び出したときの識別子表の管理の様子を示したものである．呼び出し時に退避，終了時に復元することで，各手続き内でのスコープ管理を実現している．図からわかるように，`PROC1` 呼び出し時には元のスコープ情報を引き継いでそれを拡張し，`PROC1` 終了時には `PROC0` は元のスコープ情報を復元する．したがって，最終的に `PROC1` はスコープを破棄されるため，呼び出し側の `PROC0` は `PROC1` の情報を保持することなく後続の処理を行うことができる．

図 6: 手続き呼び出し時における，識別子表の退避・復元によるスコープ管理の様子



3 実装プログラム

3.1 コード生成器の全体構成

プログラム 1 の `Compiler.run()` は、入力 `ts` の読み込みから構文解析 (AST 生成)、意味解析、CASL II プログラム生成、ファイル出力までを順に実行する制御メソッドである。 `TsSource` でトークン列を読み込み、 `Syntax.parseProgram()` により `Program` を根とする AST を得た後、 `Semantic` で宣言・スコープ・型整合性などを検査し、誤りがあればエラーを返して終了する。問題がなければ `CodeGenerator.generate(ast)` で命令列 (`List<String>`) を生成し、 `writeCas()` により `cas` へ書き出す。この際、 `writeCas()` は `lib.cas` を末尾に連結し、ライブラリルーチンを含む実行可能な CASL II プログラムとして出力する。

一方、 `CodeGenerator.generate()` はコード生成本体であり、 `emitHeader()` → `ast.accept(this)` (`Visitor` 走査) → `procOut` (手続き定義) → `emitData()` (`VAR/CHAR<i>/LIBBUF`) → `emitFooter()` (`END`) の順に命令列を組み立てて返す。(プログラム 2)

なお `ast.accept(this)` 以降は `Visitor` により AST を上位ノードから順に走査する。具体的には `Program` から開始し、まず `Block` 側 (変数宣言→手続き宣言の順) を処理した後、最後に `main` 文 (複文) を処理する流れである。このため、領域割当 (`visitVarDecl`) や手続きラベル収集・手続き本体生成 (`visitProcedureDecl`) を先に完了させた上で、メイン部の命令列が生成される。

プログラム 1: `run` メソッド

```
1 public String run(final String inputFileName, final String outputFileName) {
2     try {
3         final var in = new java.io.File(inputFileName);
4         final TsSource src = new TsSource(in);
5         final Syntax syn = new Syntax(src);
6         final Program ast = syn.parseProgram(); // 構文解析
7         if (!src.eof()) throw new ParseError(src.peek().lineNo);
8         try {
9             new Semantic(ast); // 意味解析
10        } catch (CheckError ce) {
11            return "Semantic error: line " + ce.lineNo;
12        }
13        final List<String> casLines = new CodeGenerator().generate(ast); // コード生成
14        writeCas(casLines, outputFileName);
15        return "OK";
16    } catch (ParseError e) {
17        return "Syntax error: line " + e.lineNo;
18    } catch (IOException e) {
19        return "File not found. (IOException)";
20    }
21 }
```

プログラム 2: `Codegenerator.generate` メソッド

```
1 public List<String> generate(Program ast) {
2     emitHeader();
3     ast.accept(this);
4     out.add("\tRET\t\t");
5     out.addAll(procOut);
6     emitData();
7     emitFooter();
8     return out;
9 }
```

3.2 データ領域と変数アクセスの実装

本実装では、変数名と VAR 領域上の位置の対応を `varOffset` (`Map<String,Integer>`) に保持している。具体的には、宣言処理を行う `visitVarDecl` の中で、各識別子 `name` に対して `varOffset.put(name, varCount)` を行い、その時点の `varCount` を「VAR 先頭からのオフセット」として登録する。以後の参照・代入は、必ずこの `varOffset` から得たオフセットに基づいて `VAR+offset` をアクセスする。

`visitVar` (プログラム 3) は、プログラム中の左辺変数への代入を VAR 領域への書き込みに落とし込む処理である。まず `lookupOffset(node.name())` により `varOffset` を参照し、識別子名 `node.name()` に対応するオフセット `off` を取得する。次に `LD GR2, =off` でオフセットを即値として `GR2` に設定する。右辺値は `visitAssign` が先に `rhs.accept(this)` を実行してスタックへ `PUSH` しているため、`visitVar` 側では `POP GR1` によりその値を `GR1` に取り出す。最後に `ST GR1, VAR, GR2` を発行し、`VAR+off` の 1 語へ格納することで代入を実現している。

参照側も同様に、`visitVarRef` は同じ `off` を用いて `LD ... VAR, GR2` で読み出し、`PUSH` して式へ返す。配列は `visitArrayRef/visitArrayAccess` で添字 `i` を評価した後、`k=base-lower` (`base=lookupOffset`, `lower=lookupLower`) で補正し、`ADDA GR2, =k` の後に `LD/ST` を行う。いずれもアクセスは `VAR+offset` に統一している。

プログラム 3: `visitVar` メソッド

```
1 public Void visitVar(Var node) {
2     int off = lookupOffset(node.name());
3     emit("\tLD\tGR2, =" + off + "\t; Var (" + node.name() + ")");
4     emit("\tPOP\tGR1\t; Var");
5     emit("\tST\tGR1, VAR, GR2\t; Var");
6     return null;
7 }
```

3.3 制御構文と手続き呼出の実装

• 手続き宣言におけるスコープ管理と出力先の切替

手続き本体のコード生成では、生成した命令列をメイン部とは別に蓄積するため `currentOut` を `procOut` に切り替え、手続き内でローカル変数・仮引数の登録により識別子表が拡張されるため、`varOffset`・`varKind`・`arrayLower` を退避してから生成し、終了時に復元する。これにより、手続き内で追加した識別子が外側へ漏れず、同名識別子の隠蔽を含む字句的スコープをコード生成段階で維持できる。出力先の切替と識別子表の退避・復元は、プログラム 4 に示すとおりである。

プログラム 4: `visitProcedureDecl` メソッド

```
1 public Void visitProcedureDecl(ProcedureDecl node) {
2     List<String> savedOut = currentOut; // 退避
3     currentOut = procOut; // 出力先切り替え
4
5     Map<String,Integer> savedOffset = new LinkedHashMap<>(varOffset);
6     Map<String,VarKind> savedKind = new LinkedHashMap<>(varKind);
7     Map<String,Integer> savedLower = new LinkedHashMap<>(arrayLower);
8
9     String label = procLabel.computeIfAbsent(name, n -> "PROC" + (procId++));
10    emit(label + "\tNOP\t\t; ProcedureDecl (" + name + ")"); // PROC<i> NOP
11    /* ... locals/params を登録して本体を生成 ... */
12
13    varOffset.clear(); varOffset.putAll(savedOffset); // 復元
```

```

14     varKind.clear();    varKind.putAll(savedKind);
15     arrayLower.clear(); arrayLower.putAll(savedLower);
16
17     currentOut = savedOut;
18     return null;
19 }

```

- **if/while の展開**

visitIfS と visitWhileS では、jumpId により ELSE<i>・ENDIF<i>・LOOP<i>・ENDLP<i> を一意に採番し、emit(...) で分岐命令列を組み立てる。条件式は node.cond().accept(this) で評価し（結果はスタックに積まれる）、POP → CPA GR1,=#0000 → JZE の形で「偽なら分岐」を統一している。if は thenC().accept(this) で then 部を生成し、else があれば JUMP で合流点へ飛ばしてから ELSE<i> 側を生成し、最後に ENDIF<i> を置く。while は LOOP<i> を先頭に置き、条件が偽なら ENDLP<i> へ分岐し、真なら body().accept(this) で本体を生成した後に JUMP LOOP<i> で反復を作る。

- **CALL による手続き呼出**

手続き名と入口ラベルの対応は procLabel (Map<String,String>) で管理する。宣言側 (visitProcedureDecl) で procLabel.computeIfAbsent(name, n -> "PROC" + (procId++)) により PROC<i> を割り当て、手続き先頭にラベルを出力して入口を確定させる。呼出側 (visitProcCall) では、引数を左から順に arg.accept(this) で評価してスタックに積んだ後、procLabel.get(name) で宛先ラベルを取得し、emit("\tCALL\t"+label) を出力する。この方式により、複数の手続き宣言と複数回の呼出に対して、最小限の対応表と採番 (procId) だけで一意な分岐先を確保できる。

4 考察や工夫点

- **中間表現 (AST) を用いた設計の利点**

本課題では、構文解析で生成した AST を中間表現として、意味解析とコード生成の両方で共通利用した。ノード種別 (Program, VarDecl, Binary, IfS など) に対して Visitor を適用することで、各処理をノードごとに局所化でき、生成規則の追加・修正が他部分へ波及しにくい構造になった。特に、構文 (木構造) と処理 (意味検査・コード生成) を分離できるため、機能追加やデバッグが容易になった。

- **評価規約の統一 (式評価・真偽値・制御構文の共通化)**

式評価は「評価結果をスタックに PUSH して親へ返す」という規約に統一し、二項演算は子を先に評価して値を積み、POP して演算し、結果を再度 PUSH する方式とした。また真偽値を #FFFF (真) と #0000 (偽) に正規化し、if や while では「条件式を評価→#0000 なら分岐」の形に揃えた。この統一により、ネストした式・条件式・比較演算・制御構文を同一の枠組みで展開でき、コード生成の見通しと一貫性が向上した。さらに、手続き呼出でも仮引数を VAR へコピーして以後は通常変数と同規約で扱うことで、手続き本体は「スタック=式評価・一時値用」として自由に利用できる状態を保てるようにした。

- **生成コードの可観測性 (コメント付与と段階的検証)**

生成する CASL II コードには、構文要素 (代入, if/while, 手続き呼出など) に対応したコメントやラベルを付与し、実行時に「どの高級言語構文がどの命令列へ落ちたか」を追跡しやすくした。また、emit による行単位の蓄積と、ヘッダ/メイン/手続き/データ/フッタの分割出力により、生成結果を部分的に確認しながら段階的にデバッグできるようにした。これにより、不具合が生じた場合も、原因箇所 (式評価, 分岐骨組み, 領域参照など) を生成規約の単位で切り分けやすくなった。

- データ表現とメモリアクセス規約 (VAR 基準のオフセット設計)

変数・配列・仮引数をすべて VAR 領域に割り当て、「VAR + offset」で参照・代入する規約に統一した。visitVarDecl で varOffset を構築し、配列は arrayLower に下限を保持して添字補正 (base-lower+i) により連続確保領域へアクセスする。文字列定数は strLabel で共有し、CHAR<i> ラベルとして DC 領域へ集約することで重複定義を避けた。このように参照規約を単純化したことで、コード生成は名前解決と CASL 命令列生成を明確に分離できた。

- 今後の拡張性 (最適化・機能拡張の余地)

生成器が AST を基準に実装されているため、将来的に最適化や機能拡張を差し込みやすい。例えば覗き穴最適化として、局所的な命令列をパターンマッチして短縮できる。具体例として、PUSH 0, GR1 直後の POP GR1 のような無意味な往復の削除、LD GR1, =0 に続く ADDA GR1, =k を LD GR1, =k へ畳み込み、JUMP L; L NOP のような冗長ラベルの整理などが考えられる。また、将来的にレジスタ割当や定数畳み込み、分岐簡約などを導入する場合も、「AST 段階の最適化」または「CASL 出力後の最適化」を独立パスとして追加でき、現行実装を大きく崩さずに性能改善を進められる。

5 感想

本課題では、字句解析・構文解析・意味解析で得られた情報を用いて、最終的に CASL II の命令列を生成するまでを一貫して実装した。特に、機能を追加するたびにテストケースを 1 つずつ確認し、原因を切り分けながら修正していく作業は想像以上に手間がかかり、大変であった。一方で、小さなテストを積み重ねることで不具合の発生箇所を特定しやすくなり、最終的な安定動作につながることも実感できた。

また、構文解析で作成した AST を中間表現として使い、意味解析とコード生成を Visitor パターンで分離して実装したことで、処理の責務を整理しながら拡張できた点は有用だった。ノードごとの visit に局所的な規約 (スタック規約や VAR+offset 参照規約) を持たせることで、生成規則を追跡しやすく、保守もしやすい構成になったと感じる。

さらに、生成された CASL II アセンブリコードを追いながら動作を確認する過程で、高級言語の構文が条件分岐やスタック操作、データ領域アクセスなどの具体的な命令列に落ちる様子を理解できた。コンパイラが出力したコードを手で読んで「プログラムが実際にどのように動いているか」を把握する経験は、普段のプログラミングでは得にくい学びであり、今後の低レイヤ理解やデバッグにも役立つと考える。